

การวิเคราะห์ผล การประเมินประสิทธิภาพเชิงปริมาณ การรู้จำตัวอักษร (OCR) วัด Character Error Rate (CER) ด้วย Levenshtein distance ระหว่างผลลัพธ์ของโมเดลกับข้อความอ้างอิง ดังสูตร
$$CER = \frac{S + D + I}{N}$$
 โดยที่ S , D , I คือจำนวนตัวอักษรที่แทนที่ผิด

ขาดหายไป และแทรกเกินตามลำดับ และ N คือจำนวนตัวอักษรทั้งหมดในข้อความอ้างอิง

วัด Levenshtein distance และใช้ชุดทดสอบ

ThaiOCRBench ทั้งหมด 2,808 ภาพ

จาก 13 ชุด

และ thai-ocr-evaluation ทั้งหมด

104 ภาพ ร่วมกับข้อมูลสังเคราะห์และภาพหน้าจอจริงที่ผู้วิจัยจัดทำขึ้น วัด

Word Accuracy ตามสูตร
$$Word\ Accuracy = 1 - Word\ Error\ Rate$$
 เนื่องจากผู้ใช้รับรู้ความผิดพลาดระดับคำมากกว่าระดับตัวอักษร การวัดร่วมกับ CER ช่วยให้เห็นภาพทั้งสองระดับ

ทำให้ครอบคลุมมุมมองผู้ใช้ปลายทาง

วัดเวลาประมวลผล

เฉลี่ยต่อภาพบน CPU
เนื่องจากในกรณี

การทำงานออฟไลน์

CPU เป็นตัว
กำหนดประสิทธิภาพ
หลัก ขั้นตอนการ
ทดลอง ดำเนินการ
ตามขั้นตอนต่อไปนี้:

ตั้งค่าสภาพแวดล้อม

การทดสอบโดย

กำหนดค่า \$“seed”
= 42\$ เพื่อให้
สามารถทำซ้ำได้ แล้ว
จึงเตรียมชุดข้อมูล

ทดสอบ โดยการ
ดาวน์โหลดชุดข้อมูล

ThaiOCRBench และ
thai-ocr-evaluation
(104 ภาพ) จาก

แหล่งต้นทาง เก็บใน

./data/ocr/test/
รันสคริปต์ประเมินผล

OCR ด้วยคำสั่ง
python scripts/
eval_ocr.py -

model PP-OCRv5-
mobile -dataset

thaiocrbench -
output ./results/
ocr.json -device

cpu -seed 42
(เปลี่ยน -model PP-

OCRv5-mobile เป็น
-model ThaiTrOCR

สำหรับโมเดล

ThaiTrOCR) สคริปต์
ทำงานอัตโนมัติตาม

ลำดับ: (ก) อ่านภาพที่
ละภาพด้วย OpenCV

(ข) ปรับขนาดภาพ
ตามข้อกำหนดของ

แต่ละโมเดล สำหรับ
PP-OCRv5 ใช้

512x512 px และ
ThaiTrOCR ใช้

384x384 px
โดยใช้การปรับขนาด
แบบ bilinear
interpolation (ค)
รัน OCR inference
บน CPU (ง)
เปรียบเทียบข้อความ
ส่งออกกับ ground
truth ด้วยระยะ
ทาง Levenshtein
(จ) บันทึก CER,
Word Accuracy และ
latency (ms)
ของแต่ละภาพลงไฟล์
JSON คำนวณ
ค่า CER เฉลี่ยของ
ทั้งชุดทดสอบ ทำซ้ำ
ขั้นตอนที่ 3-5 ครบ
5 รอบ แล้วบันทึก
ค่า CER ของแต่ละ
รอบ แล้วจึงรายงาน
เป็นค่าเฉลี่ยเลขคณิต
และส่วนเบี่ยง
เบนมาตรฐาน
เปลี่ยนโมเดล
เป็น ThaiTrOCR
และทำซ้ำขั้นตอน
ก่อนหน้านี้ การ
แปลภาษา (MT)
ค่า Bilingual
Evaluation
Understudy
(BLEU) วัดค่า BLEU
Score (Bilingual
Evaluation
Understudy) ซึ่ง
วัดความแม่นยำของ
การจับคู่ n -gram
ระหว่างผลลัพธ์จาก
โมเดลกับข้อความ
อ้างอิงที่แปลโดย
มนุษย์ ร่วมกับค่าปรับ
ลดความสั้น (Brevity
Penalty: BP) ตาม
สูตร $\$$ “BLEU”

= “BP” times
 $\exp(\sum w_n \log p_n)$ โดย N
= 4, p_n
คือความแม่นยำของ

n -gram, และ
“BP” คือ brevity
penalty ความเร็วใน
การแปล วัดจำนวน
tokens ต่อวินาที บน
CPU เพราะบอกว่า
มีความเร็วเพียงพอต่อ
real-time หรือไม่
ขั้นตอนการทดลอง

ดำเนินการตาม
ขั้นตอนต่อไปนี: ตั้งค่า
สภาพแวดล้อมการ
ทดสอบโดยกำหนดค่า

“seed” = 42
เพื่อให้สามารถทำซ้ำ
ได้ แล้วจึงเตรียมชุด
ข้อมูลทดสอบ โดย
การดาวน์โหลดชุด
ข้อมูล FLORES-200
ภาษาไทยและอังกฤษ

เป็นจำนวน 3,001 คู่
ประโยค แล้วเก็บไฟล์
ไว้ใน ./data/mt/
test/flores200.th

และ ./data/mt/
test/flores200.en

รันสคริปต์ประเมินผล

MT ด้วยคำสั่ง
python scripts/
eval_mt.py -
model NLLB-200-
distilled-600M -src
th -tgt en -
dataset flores200
-output ./results/
mt.json -device
cpu -beam 4 -
max_len 256 -
seed 42 โดยเปลี่ยน
-model NLLB-200-

distilled-600M เป็น
-model NLLB-200-
distilled-1.3B

สำหรับโมเดลขนาด

1.3B สคริปต์

ทำงานอัตโนมัติตาม

ลำดับดังนี้: (ก) อ่าน
ประโยคภาษาไทย

จาก flores200.th

ทีละประโยค (ข)

tokenize ด้วย

SentencePiece

tokenizer ของ

NLLB-200 (ค)

แล้วทำการ MT

inference บน CPU

ด้วย beam search

ด้วยค่าพารามิเตอร์

ดังนี้ \$“beam size”

= 4\$, \$“max

length” = 256\$

tokens, \$“length

penalty” = 1.0\$

(ง) detokenize

ข้อความที่แปลแล้ว

(จ) เปรียบเทียบ

กับ reference จาก

flores200.en เพื่อ

คำนวณ BLEU score

(ฉ) บันทึก BLEU,

ความเร็วในการแปล

และ ความหน่วงของ

แต่ละประโยคลงไฟล์

สกุล JSON คำนวณ

ค่า BLEU โดย

รวมจากชุดประเมิน

FLORES-200 ทั้งหมด

ทำซ้ำขั้นตอนที่ 3–

5 ครบ 5 รอบ

แล้วจึงรายงานเป็น

ค่าเฉลี่ยเลขคณิต

และส่วนเบี่ยง

เบนมาตรฐาน

เปลี่ยนโมเดล

เป็น NLLB-200-

distilled-1.3B และ
ทำซ้ำขั้นตอนที่ 3-
6 เพื่อเปรียบเทียบ
ประสิทธิภาพของ
ระบบโดยรวม ระยะ
เวลาหน่วยแยกตาม
ส่วนประกอบ วัด
ระยะเวลาหน่วยแยก
ตามส่วนประกอบของ
โปรแกรมด้วย `performa`
ใน `frontend (JS)`
และ `std::time` ใน
`Rust backend`
รายงานเป็น `ms`
เป็นการชี้จุดที่ทำให้
ช้าลงได้ชัดเจนที่สุด

และช่วยบอกว่าควร
ปรับปรุงส่วนไหนก่อน

`RAM` และ `Disk`
`Usage` วัด `RAM`
ในสถานะ `idle` (ไม่
แปล) และ `active`
(กำลังประมวลผล)
ด้วยเครื่องมือ `psutil`
วัดพื้นที่เก็บรวม

ทุกโมเดล `runtime`
และแอป เพราะระบบ
ต้องทำงานบนเครื่อง
ทั่วไปที่อาจมี `RAM`
จำกัด เวลาเปิดใช้งาน
ครั้งแรก นอกจากนี้
นี้ ยังวัดเวลาตั้งแต่
เปิดแอปจนพร้อม

ใช้งานครั้งแรก เพราะ
สำคัญในสถานการณ์
ฉุกเฉินที่ต้องการ
ความรวดเร็ว ขั้นตอน
การทดลอง ดำเนิน
การตามขั้นตอนต่อ

ไปนี้: ติดตั้งเครื่องมือ

`Tauri CLI v2`
(`cargo install tauri-
cli --version ^2`)
และโปรแกรม `Rust`
`1.78.0` สำหรับ สร้าง

แอปพลิเคชัน GOaT
แล้วสร้างโปรแกรม

GOaT เวอร์ชัน
ทดสอบ cargo tauri
build -config src-
tauri/tauri.conf.release
จากนั้นติดตั้งไฟล์

.msi (Windows)
หรือ .deb (Linux)

จากกระบวนการสร้าง

ก่อนเริ่มวัดผล ให้
เปิดแอป GOaT 1
ครั้งเพื่อให้ระบบสร้าง
cache และโฟลเดอร์

ข้อมูล แล้วปิดแอป

วัดเวลาเปิดใช้งานครั้ง

แรก: (ก) รีสตาร์ท

เครื่อง OS รोजন

เข้าสู่หน้าจอหลักและ

รออีก 30 วินาที

ให้โปรแกรมพื้นหลัง

โหลดเสร็จ (ข) เริ่มจับ

เวลาที่จุดเริ่มต้นของ

ฟังก์ชัน main() ใน

Rust backend ด้วย

std::time::Instant

(ค) เปิดแอป GOaT

โดย double-click

ที่ไอคอนแอป (ง)

หยุดจับเวลาเมื่อ

frontend แสดง

overlay และปุ่ม

Capture พร้อม

ใช้งาน (จ) บันทึก

เวลาในหน่วย ms (ฉ)

ทำซ้ำขั้นตอน (ก)–(จ)

ครบ 5 รอบ รายงาน

ค่าเฉลี่ยเลขคณิต

และส่วนเบี่ยงเบน

มาตรฐาน วัดระยะ

เวลาห่วงแยกตาม

ส่วนประกอบ: (ก)

เปิด GOaTแล้ว

จับภาพหน้าจอ (ข) ใช้ per

ใน frontend JS

จับเวลาตั้งแต่กด

Capture จนได้รับผลลัพธ์กลับมา (ค) ใน

Rust backend ใช้ `std::time::Instant`

จับเวลาแยก 3 จุด

ได้แก่ `ocr_start` ถึง `ocr_end`, `mt_start`

ถึง `mt_end`, `render_start` ถึง

`render_end` (ง)

บันทึก timestamp

ทุกจุดลงไฟล์ JSON

(จ) ทำซ้ำกับภาพต่าง

กัน 100 ภาพ รายงาน

ค่าเฉลี่ยเลขคณิต

และส่วนเบี่ยงเบน

มาตรฐาน ของแต่ละ

ส่วนประกอบ และ

ร้อยละของเวลา

ทั้งหมด วัด RAM

Usage: (ก) รันสคริปต์

`monitor_ram.py` ซึ่ง

เรียก `psutil.Process().m`

ที่ `sampling rate`

1 Hz (ข) สภาวะ

idle: เปิด GOaT

ทิ้งไว้ 60 วินาทีโดย

ไม่กดปุ่มใดๆ บันทึก

RAM ทุก 1 วินาที

(ค) สภาวะ active:

ส่งภาพ 1 ภาพผ่าน

pipeline ครบ 1

cycle บันทึกค่า peak

RAM ขณะประมวล

ผล (ง) รายงาน

ค่าเฉลี่ยเลขคณิต

และส่วนเบี่ยงเบน

มาตรฐาน สำหรับ

idle และ

peak value สำหรับ

active วัดพื้นที่

เก็บข้อมูลที่แอปใช้

เพื่อคำนวณขนาด

รวมของฟลैตเตอร์

แอป (รวมโมเดลและ dependencies ต่างๆ) การวิเคราะห์ทางสถิติ การทดสอบ

Paired t-test เราเปรียบเทียบคู่โมเดล

(ThaiTrOCR vs PP-OCRv5, NLLB-600M vs 1.3B, ก่อน vs หลัง fine-tune) ด้วย

paired t-test ระดับนัยสำคัญ $\alpha = 0.05$ ใช้ paired test เพราะแต่ละคู่ทดสอบบนชุดข้อมูล

ชุดเดียวกัน การหาค่า

Cohen's d รายงาน

Cohen's d

ตามสูตร $d = (\mu_1 - \mu_2) / \sigma_{\text{pooled}}$

โดย $\sigma_{\text{pooled}} = \sqrt{(\sigma_1^2 + \sigma_2^2) / 2}$

แปลผลตามเกณฑ์

ของ Cohen: $d \geq 0.2$ เล็ก, $d \geq 0.5$ กลาง,

$d \geq 0.8$ ใหญ่ รายงาน

95% CI ของทุกเกณฑ์การวัดคำนวณจาก

bootstrap 10,000 รอบ เพื่อให้ค่า

ประมาณความไม่

แน่นอน การประเมินประสิทธิภาพเชิง

คุณภาพ สภาพแวดล้อมในการ

ประเมิน ใช้ผู้ประเมิน

3 คนที่เป็นเจ้าของภาษาไทยและคนที่พูดอังกฤษได้ เนื่องจากจำนวน 3 คนเป็นจำนวนขั้นต่ำที่ให้ผล

การรายงานที่เชื่อถือ
ได้ สุ่มลำดับ 300
ตัวอย่าง แบบปกปิด
ว่าเป็นของโปรแกรม
GOaT หรือ baseline
การให้คะแนน Direct
Assessment วัด
ค่า Adequacy
(0-100) เพื่อแปลผล
ว่าความหมายตรงกับ
ต้นทางหรือไม่ และ
Fluency (0-100)
เพื่อแปลผลว่าภาษา
ไทยเป็นธรรมชาติ
หรือไม่ ทั้งสองอย่าง
นี้ครอบคลุมทั้งความ
ถูกต้องเชิงความหมาย
และความเป็น
ธรรมชาติของภาษา
การวิเคราะห์ประเภท
ความผิดพลาด (Error
Categorization)
แยกตามสาเหตุ

3 ประเภท:
Type A สำหรับ
OCR ผิดพลาดเท่านั้น
Type B สำหรับ MT
ผิดพลาดเท่านั้น และ
Type C ผิดพลาด
ทั้งคู่ นับจำนวนและ
ร้อยละของตัวอย่าง
ในแต่ละประเภท

ขั้นตอนการดำเนินการ
ประเมิน ดำเนิน
การตามขั้นตอนต่อไป
นี้: คัดเลือกผู้ประเมิน

3 คน ที่เป็นเจ้าของ
ภาษาไทยและคนที่
พูดอังกฤษได้ ไม่ใช่ผู้
พัฒนา GOaT และไม่เคย
เห็นระบบมาก่อน

จัดเตรียมตัวอย่าง
ทดสอบ 300 ตัวอย่าง

โดยสุ่มจากภาพหน้า
จอจริง 500 ภาพ

จากนั้นสุ่มลำดับการ
นำเสนอแบบปกปิด
ว่าเป็นของโปรแกรม

GOaT หรือ
baseline จัดทำ
แบบประเมิน แต่ละ
ตัวอย่างประกอบ

ด้วย: (ก) ภาพ
หน้าจอต้นฉบับ (ข)
ข้อความที่ระบบแปล

ได้ (ค) reference
translation โดย
ผู้เชี่ยวชาญ (ง) ช่อง

คะแนน Adequacy
0-100 (เพื่อแปลผล
ว่าความหมายตรงกับ

ต้นทางหรือไม่) (จ)
ช่องคะแนน Fluency
0-100 (เพื่อแปลผล

ว่าภาษาไทยเป็น
ธรรมชาติหรือไม่) จัด
calibration session

ก่อนเริ่มประเมินจริง
: ผู้ประเมินทั้ง 3
คนให้คะแนนตัวอย่าง

10 ตัวอย่างรวม
กัน อภิปรายความ
แตกต่างของคะแนน

เพื่อปรับมาตรฐาน
เดียวกัน ใช้เวลา 30
นาที ผู้ประเมินแต่ละ

คนให้คะแนน 300
ตัวอย่างอิสระต่อกัน
โดยไม่ปรึกษากัน

ใช้เวลาไม่เกิน 2
ชั่วโมง พักทุก 50
ตัวอย่างเพื่อลดความ

เหนื่อยล้า รวบรวม
คะแนน วิเคราะห์
inter-annotator

agreement ด้วย
Kendall's τ_b
หาก $\tau_b < 0.5$
ให้จัด calibration
session เพิ่มเติมและ

ประเมินซ้ำ คัดแยก
ชนิดความผิดพลาด

: (ก) นำตัวอย่าง
ที่ \$“Adequacy”

< 80\$ หรือ

\$“Fluency” <

80\$ มาวิเคราะห์

(ข) ผู้ประเมิน

จำแนกสาเหตุตาม

เกณฑ์ Type A Type

B หรือ Type C (ค) ผู้

วิจัยตรวจทานซ้ำเพื่อ

ยืนยันการจำแนก (ง)

นับจำนวนและร้อยละ

ของตัวอย่างในแต่ละ

ประเภท รายงานผล

โดยรวม การวิเคราะห์

เปรียบเทียบ เทียบ

กับการทำงานแยก

ขั้นตอน เปรียบเทียบ

GOaT กับการทำงาน

ที่ใช้ OCR tool

(PaddleOCR CLI)

และ MT tool

(TranslateLocally)

แยกกันคน

ละโปรแกรม วัดเวลา

ตั้งแต่เริ่มจัดภาพ

หน้าจอจนได้ค่า

แปล และวัดจำนวน

ขั้นตอนที่ผู้ใช้ต้องทำ

ขั้นตอนการทดลอง

เปรียบเทียบ ดำเนิน

การตามขั้นตอน

ต่อไปนี้: ตั้งค่าการ

ทดสอบเทียบกับการ

ทำงานแยกขั้นตอน

: เปิด PaddleOCR

CLI ด้วยคำสั่ง

paddleocr -

image input.png

-lang th -

use_angle_cls true

-use_gpu false จาก

นั้นคัดลอกข้อความที่

OCR ได้ แล้วจึงเปิด
TranslateLocally
จากนั้นจึงวาง
ข้อความ แล้วกดปุ่ม
Translate แล้ว
คัดลอกผลลัพธ์การ
แปล จับเวลาตั้งแต่
เริ่มเปิด PaddleOCR
CLI จนได้ผลลัพธ์การ
แปล พร้อมนับจำนวน
ขั้นตอนทั้งหมดที่ผู้
ใช้ต้องทำ บันทึกผล
ทำซ้ำ 5 รอบกับ
ภาพต่างกัน 5 ภาพ
วัดเวลาการทำงาน
ของ GOaT: เปิด
GOaT แล้วกดปุ่ม
Capture และอ่านผล
จาก overlay จับเวลา
ตั้งแต่กด Capture
จน overlay แสดง
ผลการแปลครบถ้วน
บันทึกผล ทำซ้ำ
5 รอบกับภาพชุด
เดียวกับข้อก่อนหน้า
เปรียบเทียบเวลา
เฉลี่ยและจำนวน
ขั้นตอนระหว่าง
Separate-tool กับ
GOaT ด้วย
การทดสอบ paired
t-test ($\alpha = 0.05$) แล้ว
รายงานเวลาที่ลดลง
และขั้นตอนที่ลดลง